C++ STL

速通版



春云暮雪 [1]

鱼组 Fish Group

February 11, 2025

[1]关注鱼组谢谢喵

目录

1. STL
2. 命名空间
3. 算法
4. 容器
5. 迭代器
6. 作业



STL

STL 是在 C++ 中提供的一个模板库 Standard Template Library (标准模板库)。STL 包含一系列算法和容器等，合理地使用 STL，可以在提高编写代码的效率。

STL

STL 是在 C++ 中提供的一个模板库 Standard Template Library (标准模板库)。STL 包含一系列算法和容器等，合理地使用 STL，可以在提高编写代码的效率。

STL 中提供了六大部分：
容器（containers） 迭代器（iterators） 空间配置器（allocator） 配接器（adapters） 算法（algorithms） 仿函数（functors）六个部分。

STL

STL 是在 C++ 中提供的一个模板库 Standard Template Library (标准模板库)。STL 包含一系列算法和容器等，合理地使用 STL，可以在提高编写代码的效率。

STL 中提供了六大部分：
容器（containers） 迭代器（iterators） 空间配置器（allocator） 配接器（adapters） 算法（algorithms） 仿函数（functors）六个部分。

其中比较常见需要了解掌握的是三个部分：迭代器（iterators） 算法（algorithms） 容器（containers）

命名空间

命名空间（namespace）是 C++ 的一个特性，它被用于解决名称冲突，比如假设 山田リョウ 和 下北泽大天使いじちにじか 都在自己的头文件里编写了一个 Music() 函数，如果这两个函数我们都要用，则名称会产生冲突，但如果两人都将自己的函数写在单独的命名空间内，就成了山田::Music() 和虹夏::Music()，就不会冲突了。

命名空间

STL 的所有内容都包含在 `std` 命名空间内。

命名空间

STL 的所有内容都包含在 `std` 命名空间内。
如果我们需要调用 STL 中的`sort`函数，我们需要这样写：

```
std::sort(a,a + n);
```

命名空间

STL 的所有内容都包含在 `std` 命名空间内。
如果我们需要调用 STL 中的`sort`函数，我们需要这样写：

```
std::sort(a,a + n);
```

我们也可以将 `std::sort` 这个函数“导入”到全局中，就可以直接 `sort(a,a + n)` 这样调用了。使用 `using` 关键字来“导入”命名空间中的函数或类。

命名空间

STL 的所有内容都包含在 `std` 命名空间内。
如果我们需要调用 STL 中的`sort`函数，我们需要这样写：

```
std::sort(a,a + n);
```

我们也可以将 `std::sort` 这个函数“导入”到全局中，就可以直接 `sort(a,a + n)` 这样调用了。使用 `using` 关键字来“导入”命名空间中的函数或类。

```
using std::sort;
sort(a,a + n);
```

命名空间

当然你也可以直接导入整个 std 的命名空间
但这样就会出现比如你写了一个变量 max 和命名空间内部函数相撞的情况

```
using namespace std;
```

算法

STL 中附带了很多非常好用的算法，大部分位于include<algorithm>下，以下我挑几个讲讲

想完备了解的请查看 参考手册

算法

`sort()`：排序函数，采用的是随机快速排序方法，在不构造特殊数据的情况下，应该跑的比大部分人手写的排序快很多，而且我猜这应该是最常用的函数之一了

`sort(begin,end,cmp)`

会将 begin 和 end 中的地址元素按照 cmp 的方法进行比较

不传入 cmp 时默认从小到大

算法

sort() 可以排序任意数据类型，前提是需要传入比较函数cmp() 或重载数据类型的小于号

```
1 struct P{int x,int y}A[10];
2 bool cmp(P X,P Y){return X.x < Y.x;}
3 bool operator < (P X,P Y){return X.x < Y.x;}
4 sort(A,A + 10,cmp);
5 sort(A,A + 10);
```

算法

unique 去除容器中相邻的重复元素，所以如果要去重所有元素，需要所有重复元素相邻，推荐排序使用

unique(begin,end)，他的去除是将重复的数字向后挪，最后返回去重后容器结尾的结尾

搭配sort 使用，可以完成对一个数组或容器的去重化

```
1 int a[10];
2 // ....
3 sort(a,a + 10);
4 int cnt = unique(a,a + 10) - a;
```

请尝试通过这个题 第 k 小整数

算法

`lower_bound(begin,end,x)`在一个**有序序列**中进行二分查找，返回指向第一个 **大于等于** x 的元素的位置的地址。

`upper_bound(begin,end,x)`，在一个**有序序列**中进行二分查找，返回指向第一个 **大于** x 的元素的位置的地址。

注意

在一般数组里这两个函数的复杂度都是 $O(\log n)$

但是如果在 *set* 等一些关联容器中直接使用，复杂度为 $O(n)$

set 等关联容器中自带这两个函数，通过 `s.lower_bound(val)` 类型的方法使用，复杂度为 $O(\log n)$

算法

random_shuffle(begin,end) 将序列随机打乱

next_permutation(begin,end) 将一个全排列生成他的下一个排列

prev_permutation(begin,end) 将一个全排列生成他的下一个排列

注意

在 C++14 及以后的标准中，请使用

```
1 #include <random>
2 std::mt19937 rng(std::random_device{}());
3 std::shuffle(v.begin(), v.end(), rng);
```

Vector

STL 在头文件 `#include<vector>` 中提供了一个序列式的容器 *vector*

Vector

STL 在头文件 `#include<vector>` 中提供了一个序列式的容器 *vector*

使用 `vector<T>a;` 会创建一个初始元素长度为 0 的动态数组

Vector

STL 在头文件 `#include<vector>` 中提供了一个序列式的容器 *vector*

使用 `vector<T>a;` 会创建一个初始元素长度为 0 的动态数组

使用 `vector<T>a(n);` 会创建一个初始元素长度为 n 的动态数组

Vector

STL 在头文件 `#include<vector>` 中提供了一个序列式的容器 *vector*

使用 `vector<T>a;` 会创建一个初始元素长度为 0 的动态数组

使用 `vector<T>a(n);` 会创建一个初始元素长度为 n 的动态数组

使用 `a.size()` 获得 a 的元素数量

Vector

STL 在头文件 `#include<vector>` 中提供了一个序列式的容器 *vector*

使用 `vector<T>a;` 会创建一个初始元素长度为 0 的动态数组

使用 `vector<T>a(n);` 会创建一个初始元素长度为 n 的动态数组

使用 `a.size()` 获得 a 的元素数量

使用 `a.empty()` 返回 a 是否为空

Vector

STL 在头文件 `#include<vector>` 中提供了一个序列式的容器 *vector*

使用 `vector<T>a;` 会创建一个初始元素长度为 0 的动态数组

使用 `vector<T>a(n);` 会创建一个初始元素长度为 n 的动态数组

使用 `a.size()` 获得 a 的元素数量

使用 `a.empty()` 返回 a 是否为空

使用 `a.push_back(X)` 和 `a.pop_back()` 会在 a 的末尾插入或删除一个元素, 复杂度 $O(1)$

Vector

STL 在头文件 `#include<vector>` 中提供了一个序列式的容器 *vector*

使用 `vector<T>a;` 会创建一个初始元素长度为 0 的动态数组

使用 `vector<T>a(n);` 会创建一个初始元素长度为 n 的动态数组

使用 `a.size()` 获得 a 的元素数量

使用 `a.empty()` 返回 a 是否为空

使用 `a.push_back(X)` 和 `a.pop_back()` 会在 a 的末尾插入或删除一个元素, 复杂度 $O(1)$

特殊的，当你定义的类的元素是 *pair* 或其他类似的类型，你可以使用`a.emplace_back(v_1,v_2,...)` 来直接插入一个元素为 $v_1, v_2, \ldots$ 的 *pair* 等类型 (需要 C++11)

Vector

STL 在头文件 `#include<vector>` 中提供了一个序列式的容器 *vector*

使用 `vector<T>a;` 会创建一个初始元素长度为 0 的动态数组

使用 `vector<T>a(n);` 会创建一个初始元素长度为 n 的动态数组

使用 `a.size()` 获得 a 的元素数量

使用 `a.empty()` 返回 a 是否为空

使用 `a.push_back(X)` 和 `a.pop_back()` 会在 a 的末尾插入或删除一个元素, 复杂度 $O(1)$

特殊的，当你定义的类的元素是 *pair* 或其他类似的类型，你可以使用`a.emplace_back(v_1,v_2,...)` 来直接插入一个元素为 $v_1, v_2, \ldots$ 的 *pair* 等类型 (需要 C++11)

使用`a.insert(place,X)` 和 `a.erase(place,X)`可以在特定位置插入和删除一个元素，复杂度 $O(n)$ 在理论上可以替代传统的静态数组使用

Vector

以下看一个示例

Queue

我跳过了栈，因为栈手写在各方面完全比 STL 自带的厉害。

STL 在头文件 `#include<queue>` 中提供了一个序列式的队列容器 *queue*

Queue

我跳过了栈，因为栈手写在各方面完全比 STL 自带的厉害。

STL 在头文件 `#include<queue>` 中提供了一个序列式的队列容器 *queue*

使用 `queue<T>a;` 会创建一个队列

Queue

我跳过了栈，因为栈手写在各方面完全比 STL 自带的厉害。

STL 在头文件 `#include<queue>` 中提供了一个序列式的队列容器 *queue*

使用 `queue<T>a;` 会创建一个队列

使用 `a.push(X)` 将一个元素插入到队列中

Queue

我跳过了栈，因为栈手写在各方面完全比 STL 自带的厉害。

STL 在头文件 `#include<queue>` 中提供了一个序列式的队列容器 *queue*

使用 `queue<T>a;` 会创建一个队列

使用 `a.push(X)` 将一个元素插入到队列中

使用 `a.front()` 查询队列队首元素

Queue

我跳过了栈，因为栈手写在各方面完全比 STL 自带的厉害。

STL 在头文件 `#include<queue>` 中提供了一个序列式的队列容器 *queue*

使用 `queue<T>a;` 会创建一个队列

使用 `a.push(X)` 将一个元素插入到队列中

使用 `a.front()` 查询队列队首元素

使用 `a.pop()` 删除队列队首元素

Queue

我跳过了栈，因为栈手写在各方面完全比 STL 自带的厉害。

STL 在头文件 `#include<queue>` 中提供了一个序列式的队列容器 *queue*

使用 `queue<T>a;` 会创建一个队列

使用 `a.push(X)` 将一个元素插入到队列中

使用 `a.front()` 查询队列队首元素

使用 `a.pop()` 删除队列队首元素

当你在写 BFS 的时候可能会用到

Priority queue

STL 在头文件 `#include<queue>` 中提供了一个红黑树实现的关联式的优先队列容器 *priorityqueue*

Priority queue

STL 在头文件 `#include<queue>` 中提供了一个红黑树实现的关联式的优先队列容器 *priorityqueue*

大家都看到了，这个词非常的长，如果大家不想记住，我可以教大家怎么找这个词。

Priority queue

STL 在头文件 `#include<queue>` 中提供了一个红黑树实现的关联式的优先队列容器 *priorityqueue*

大家都看到了，这个词非常的长，如果大家不想记住，我可以教大家怎么找这个词。

使用 `priority_queue<T>a;` 会创建一个优先队列，元素类型需要有偏序定义

Priority queue

STL 在头文件 `#include<queue>` 中提供了一个红黑树实现的关联式的优先队列容器 *priorityqueue*

大家都看到了，这个词非常的长，如果大家不想记住，我可以教大家怎么找这个词。

使用 `priority_queue<T>a;` 会创建一个优先队列，元素类型需要有偏序定义

使用 `a.push(X)` 将一个元素插入到优先队列中

Priority queue

STL 在头文件 `#include<queue>` 中提供了一个红黑树实现的关联式的优先队列容器 *priorityqueue*

大家都看到了，这个词非常的长，如果大家不想记住，我可以教大家怎么找这个词。

使用 `priority_queue<T>a;` 会创建一个优先队列，元素类型需要有偏序定义

使用 `a.push(X)` 将一个元素插入到优先队列中

使用 `a.top()` 查询优先队列队首元素

Priority queue

STL 在头文件 `#include<queue>` 中提供了一个红黑树实现的关联式的优先队列容器 *priorityqueue*

大家都看到了，这个词非常的长，如果大家不想记住，我可以教大家怎么找这个词。

使用 `priority_queue<T>a;` 会创建一个优先队列，元素类型需要有偏序定义

使用 `a.push(X)` 将一个元素插入到优先队列中

使用 `a.top()` 查询优先队列队首元素

当你在优化 dij 最短路的时候会用到

Deque

STL 在头文件 `#include<queue>` 中提供了一个双端队列容器 *deque* , 我建议大家不要使用这个，原因我后面再说

Deque

STL 在头文件 `#include<queue>` 中提供了一个双端队列容器 *deque* , 我建议大家不要使用这个，原因我后面再说

使用 `deque<T>a;` 会创建一个双端队列

Deque

STL 在头文件 `#include<queue>` 中提供了一个双端队列容器 *deque* , 我建议大家不要使用这个，原因我后面再说

使用 `deque<T>a;` 会创建一个双端队列

使用 `a.push_back(X)` 和 `a.push_front(X)`将一个元素插入到双端队列的队尾和队首

Deque

STL 在头文件 `#include<queue>` 中提供了一个双端队列容器 *deque* , 我建议大家不要使用这个，原因我后面再说

使用 `deque<T>a;` 会创建一个双端队列

使用 `a.push_back(X)` 和 `a.push_front(X)`将一个元素插入到双端队列的队尾和队首

使用 `a.back()` 和 `a.front(X)`获取队尾和队首元素

注意

deque 所需的空间和时间都较大，我有一位友人在 NOI 上因为狂开 deque, 收获 MLE，最终痛失金牌，警钟长鸣

注意

deque 所需的空间和时间都较大，我有一位友人在 NOI 上因为狂开 deque, 收获 MLE，最终痛失金牌，警钟长鸣

对于一些容器可以使用`a.clear()`用复杂度 $O(n)$ 来清空容器

注意

deque 所需的空间和时间都较大，我有一位友人在 NOI 上因为狂开 deque, 收获 MLE，最终痛失金牌，警钟长鸣

对于一些容器可以使用`a.clear()`用复杂度 $O(n)$ 来清空容器

但可以有如下操作
直接开一个空的容器，然后使用 $swap(a, b)$, 把空容器 b 交换给 a，这样可以使用 $O(1)$ 来清空容器 a ，一般建议直接清空，不建议这样用.(队列相关的容器需要 C++11)

Set

STL 在头文件 `#include<set>` 中提供了一个使用红黑树构造的关联式的集合容器 *set*

Set

STL 在头文件 `#include<set>` 中提供了一个使用红黑树构造的关联式的集合容器 *set*

使用 `set<T>a;` 会创建一个集合

值得注意的是元素类型需要拥有偏序定义

Set

STL 在头文件 `#include<set>` 中提供了一个使用红黑树构造的关联式的集合容器 *set*

使用 `set<T>a;` 会创建一个集合

值得注意的是元素类型需要拥有偏序定义

使用`a.insert(X)` 和 `a.erase(X)`可以往集合中插入和删除一个元素，复杂度 $O(\log n)$

Set

STL 在头文件 `#include<set>` 中提供了一个使用红黑树构造的关联式的集合容器 *set*

使用 `set<T>a;` 会创建一个集合

值得注意的是元素类型需要拥有偏序定义

使用`a.insert(X)` 和 `a.erase(X)`可以往集合中插入和删除一个元素，复杂度 $O(\log n)$

`size` 和 `empty`等一些函数都和 *vector* 类似, 但他不能通过下标操作符任意访问，他的访问必须是通过迭代器顺序的访问

Set

STL 在头文件 `#include<set>` 中提供了一个使用红黑树构造的关联式的集合容器 *set*

使用 `set<T>a;` 会创建一个集合

值得注意的是元素类型需要拥有偏序定义

使用`a.insert(X)` 和 `a.erase(X)`可以往集合中插入和删除一个元素，复杂度 $O(\log n)$

`size` 和 `empty`等一些函数都和 *vector* 类似, 但他不能通过下标操作符任意访问，他的访问必须是通过迭代器顺序的访问

set 中不允许有重复元素，如果插入重复元素可能会出现 RE 问题，为了解决这个问题有 *Multiset*, 请自行搜索

Map

STL 在头文件 `#include<map>` 中提供了一个使用红黑树构造的关联式的映射容器 *map*

Map

STL 在头文件 `#include<map>` 中提供了一个使用红黑树构造的关联式的映射容器 *map*

使用`map<T1,T2>a;`来定义一个映射 a

其中元素类型 T1 需要有偏序定义

Map

STL 在头文件 `#include<map>` 中提供了一个使用红黑树构造的关联式的映射容器 *map*

使用`map<T1,T2>a;`来定义一个映射 a

其中元素类型 T1 需要有偏序定义

使用`a[x]` 来取出对应映射，其中 x 是 T1 类型，返回值为 T2 类型

Map

STL 在头文件 `#include<map>` 中提供了一个使用红黑树构造的关联式的映射容器 *map*

使用`map<T1,T2>a;`来定义一个映射 a

其中元素类型 T1 需要有偏序定义

使用`a[x]` 来取出对应映射，其中 x 是 T1 类型，返回值为 T2 类型

值得注意的是如果你直接使用 `a[x]`，而 `a[x]` 在之前并没有被赋值过，它会自动创建一个元素，如果你不希望这样，你需要使用如下语句

Map

STL 在头文件 `#include<map>` 中提供了一个使用红黑树构造的关联式的映射容器 *map*

使用`map<T1,T2>a;`来定义一个映射 a

其中元素类型 T1 需要有偏序定义

使用`a[x]` 来取出对应映射，其中 x 是 T1 类型，返回值为 T2 类型

值得注意的是如果你直接使用 `a[x]`，而 `a[x]` 在之前并没有被赋值过，它会自动创建一个元素，如果你不希望这样，你需要使用如下语句

```
if(a.find(X) != a.end())
```

Unorder 类

在前面的提到的 *set*,*map* 等类型中，他们的底层结构是一种平衡树红黑树，C++ 在后期的一些标准中还提供使用 *Hash* 完成的类型，使用这些类型不需要有偏序关系

Unorder 类

在前面的提到的 *set*,*map* 等类型中，他们的底层结构是一种平衡树红黑树，C++ 在后期的一些标准中还提供使用 *Hash* 完成的类型，使用这些类型不需要有偏序关系

详情查看 无序关联容器

pair

STL 在头文件 `#include<pair>` 中提供了一个捆绑类型类

pair

STL 在头文件 `#include<pair>` 中提供了一个捆绑类型类

可以使用`pair<T1,T2>`写出一个捆绑类型，他自带偏序定义，即先比较第一位，再比较第二位, 该类型可以搭配其他容器使用

pair

STL 在头文件 `#include<pair>` 中提供了一个捆绑类型类

可以使用`pair<T1,T2>`写出一个捆绑类型，他自带偏序定义，即先比较第一位，再比较第二位, 该类型可以搭配其他容器使用

通过 `make_pair(X,Y)` 将两个数据捆绑起来

pair

STL 在头文件 #include<pair> 中提供了一个捆绑类型类

可以使用pair<T1,T2>写出一个捆绑类型，他自带偏序定义，即先比较第一位，再比较第二位, 该类型可以搭配其他容器使用

通过 make_pair(X,Y) 将两个数据捆绑起来

通过 X.first() 和 X.second() 来取出第一位和第二位

pair

STL 在头文件 #include<pair> 中提供了一个捆绑类型类

可以使用pair<T1,T2>写出一个捆绑类型，他自带偏序定义，即先比较第一位，再比较第二位, 该类型可以搭配其他容器使用

通过 make_pair(X,Y) 将两个数据捆绑起来

通过 X.first() 和 X.second() 来取出第一位和第二位

我们来看个实例代码

String

STL 在头文件 #include<string> 中提供了一个字符串类

String

STL 在头文件 `#include<string>` 中提供了一个字符串类

他和字符数组的最大差别是重载了加号为字符串拼接，重载了小于号为字符串字典序比较，同时他是动态的，很好用。

String

STL 在头文件 #include<string> 中提供了一个字符串类

他和字符数组的最大差别是重载了加号为字符串拼接，重载了小于号为字符串字典序比较，同时他是动态的，很好用。

注意

使用strlen(a)计算一遍 *string* 类的长度复杂度为 $O(n)$

最典的错误就是for(int i = 0;i < strlen(a);++i)

所以请提前开一个变量存储，或者使用a.size() 或 a.length()

迭代器

这里仅简单的介绍一下迭代器和一些他的用法，理论上你可以手写任何数据类型的迭代器，但这里不深入探讨。

迭代器

这里仅简单的介绍一下迭代器和一些他的用法，理论上你可以手写任何数据类型的迭代器，但这里不深入探讨。

你可以把迭代器简单的看成是指针类型, 他支持`x++`,`x--`操作

迭代器

这里仅简单的介绍一下迭代器和一些他的用法，理论上你可以手写任何数据类型的迭代器，但这里不深入探讨。

你可以把迭代器简单的看成是指针类型, 他支持`x++`,`x--`操作

迭代器的类型定义如下`T::iterator X`

迭代器

这里仅简单的介绍一下迭代器和一些他的用法，理论上你可以手写任何数据类型的迭代器，但这里不深入探讨。

你可以把迭代器简单的看成是指针类型, 他支持`x++`,`x--`操作

迭代器的类型定义如下`T::iterator X`

以上所有的 STL 容器都可以通过迭代器来进行遍历，任意容器的迭代器起点为 `a.begin()`, 任意容器的迭代器尾部为 `a.end()`

迭代器

这里仅简单的介绍一下迭代器和一些他的用法，理论上你可以手写任何数据类型的迭代器，但这里不深入探讨。

你可以把迭代器简单的看成是指针类型, 他支持x++,x--操作

迭代器的类型定义如下T::iterator X

以上所有的 STL 容器都可以通过迭代器来进行遍历，任意容器的迭代器起点为 a.begin(), 任意容器的迭代器尾部为 a.end()

你可以使用如下语句遍历整个容器

```
for(T::iterator x = a.begin();x != a.end();++a){//....
*x}
```

请自动把类型 T 换掉
类似于

```
for(vector<int>::iterator x = a.begin();x != a.end();++
a){//.... *x}
```

迭代器

更多用法请查看迭代器

auto

在 C++14 标准后，可以使用如下语句

```
for(auto x : a){//.... x}
```

他会按照顺序来访问容器 a，但只有遍历有用，在一些位置访问上还是迭代器比较厉害

珂朵莉树

好的，你已经学会了加减法了，请来试试**有限域上多项式因子分解的 Cantor–Zassenhaus 过程以及 Kedlaya–Umans 优化**吧

珂朵莉树

弄错了，作业不是上面那个

珂朵莉树

请自行尝试实现如下功能：

维护不同颜色的区间段

支持区间颜色修改

修改后相邻的相同颜色区间合并

输出所有区间

珂朵莉树

即给定 n 个位置，每个位置有一个颜色，初始时位置均为 1

支持如下操作

将 $[l, r]$ 位置上的颜色修改为 x

按示意输出位置信息

珂朵莉树

例如目前拥有 10 个位置的序列，其位置上对应的颜色为
1, 1, 2, 2, 3, 4, 2, 2, 3, 3

请输出

[1,2] is 1
[3,4] is 2
[5,5] is 3
[6,6] is 4
[7,8] is 2
[9,10] is 3

请使用自行定义的结构体的 *set* 类型，迭代器来实现，在颜色修改时将原区间先分裂，再将相邻的区间合并
有点难度，大家加油